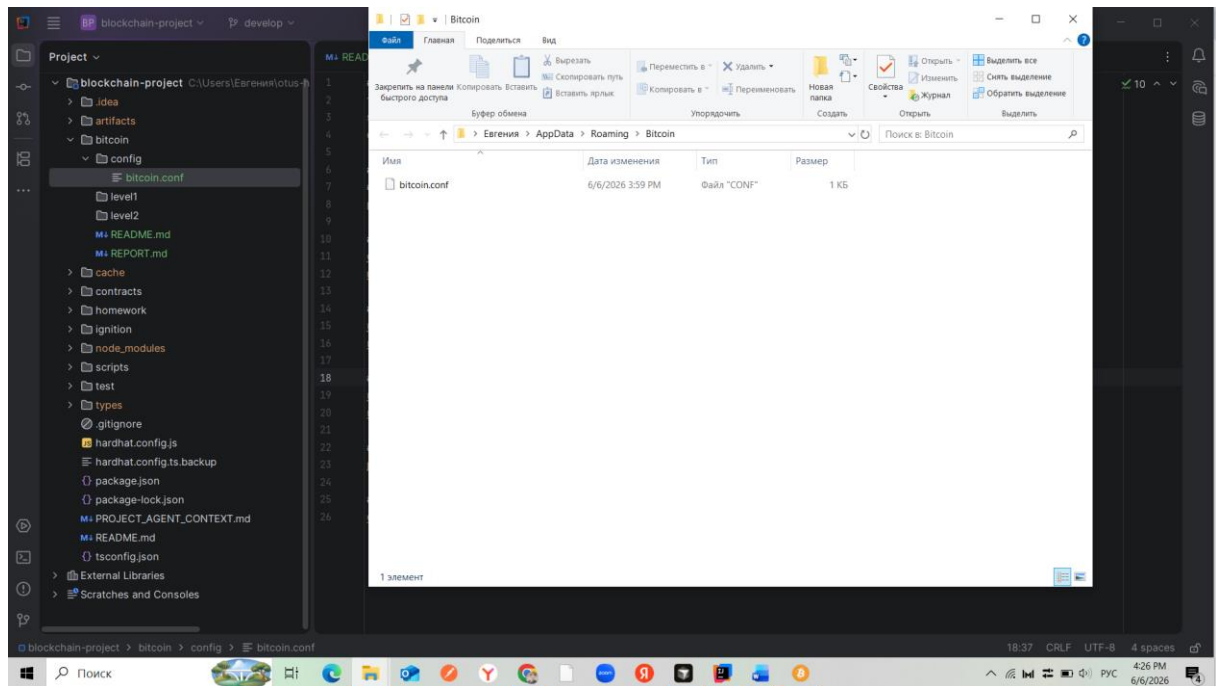
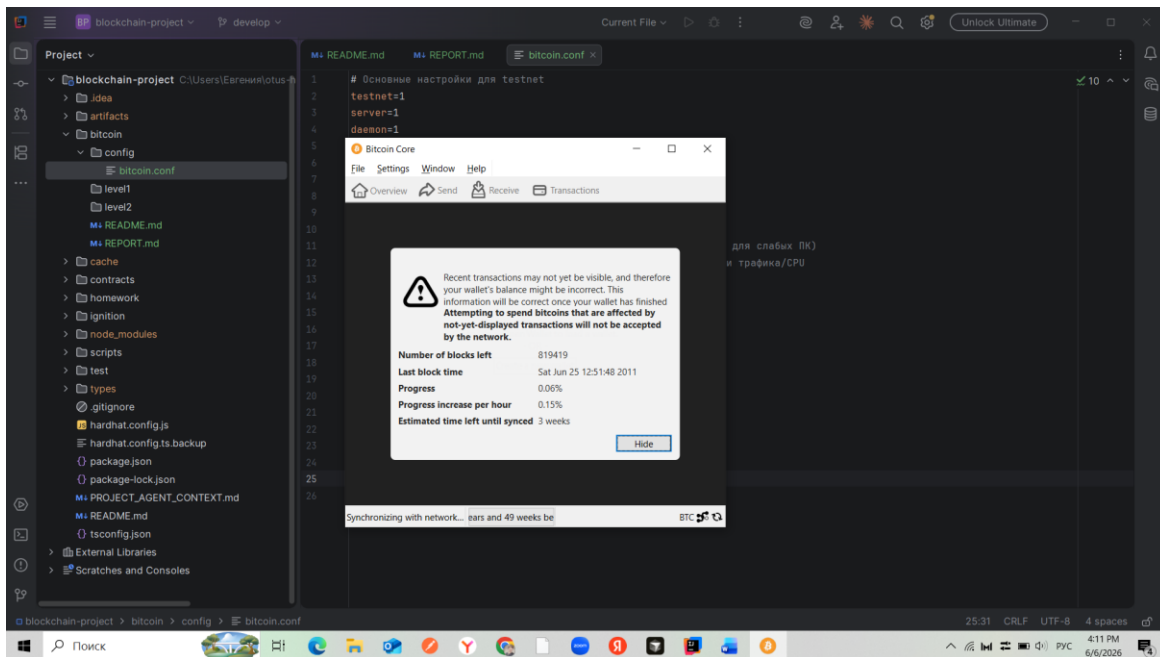


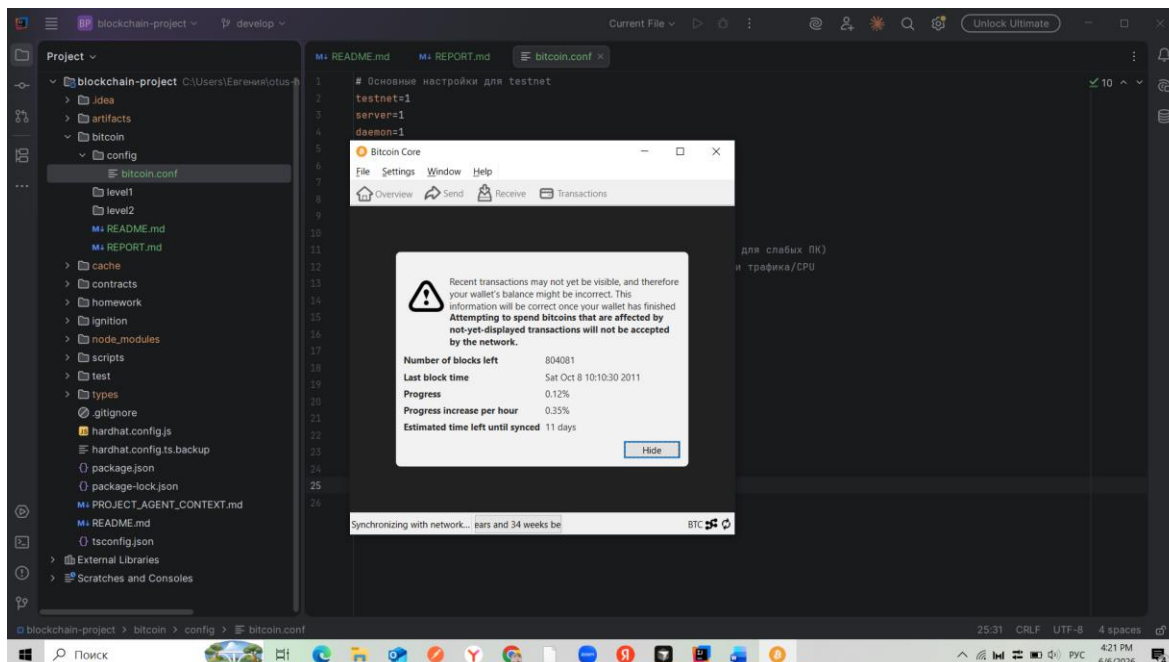
Предприняла попытку выполнить задание для Уровня 2:

1. Установила Bitcoin Core
2. Скопировала на свой диск C файл из своего проекта (запушила в GitHub) bitcoin/config/bitcoin.conf



3. Запустила Bitcoin Core
4. Вижу вычитку блоков:





Из-за недостаточной мощности ноутбука и невысокой скорости интернета вынуждена отказаться от варианта ДЗ для уровня 2 – слишком долго ждать окончания синхронизации.

Переключилась на ДЗ Уровень 1 – см.ниже.

Level 1

1. Выбор и настройка подключения

- Выбрала публичный API-провайдер **Blockstream Esplora**
- Выполнила HTTP-запрос на получение информации о последнем блоке:
<https://blockstream.info/testnet/api/blocks/tip/hash>
- Вижу хэш последнего блока:



2. Исследование функциональности

- Опишите функции узла Bitcoin: проверка транзакций, валидация блоков, mempool, пиры:

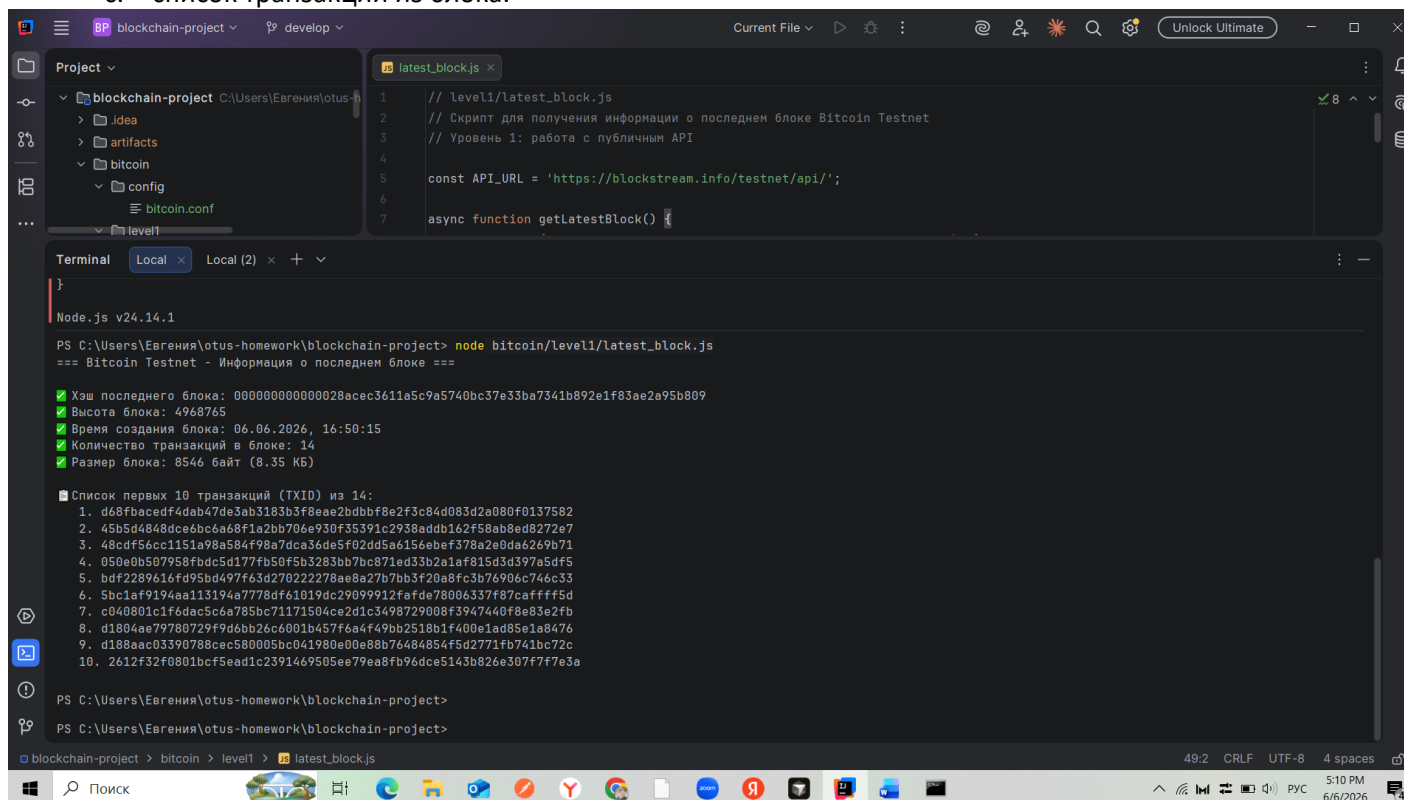
Функция	Что делает	Почему важна
Проверка транзакций	Проверяет подписи, отсутствие двойной траты, соответствие формату	Предотвращает мошенничество и повторную трату монет
Валидация блоков	Проверяет Proof-of-Work и все транзакции в новом блоке	Обеспечивает целостность блокчейна и консенсус
Mempool	Хранит неподтверждённые транзакции в очереди перед включением в блок	Позволяет майнерам выбирать транзакции по комиссии, а отправителям — отслеживать статус

Функция	Что делает	Почему важна
Пирь	Узлы обмениваются данными с другими узлами P2P-сети	Обеспечивает децентрализацию и быстрое распространение информации

3. Взаимодействие через JSON-RPC/API

Напишите скрипт (Python/JS), который выводит:

- хэш последнего блока и его высоту,
- данные о последнем блоке (время, кол-во транзакций, размер),
- список транзакций из блока.



```

1 // level1/latest_block.js
2 // Скрипт для получения информации о последнем блоке Bitcoin Testnet
3 // Уровень 1: работа с публичным API
4
5 const API_URL = 'https://blockstream.info/testnet/api/';
6
7 async function getLatestBlock() {
8
9 }

```

```

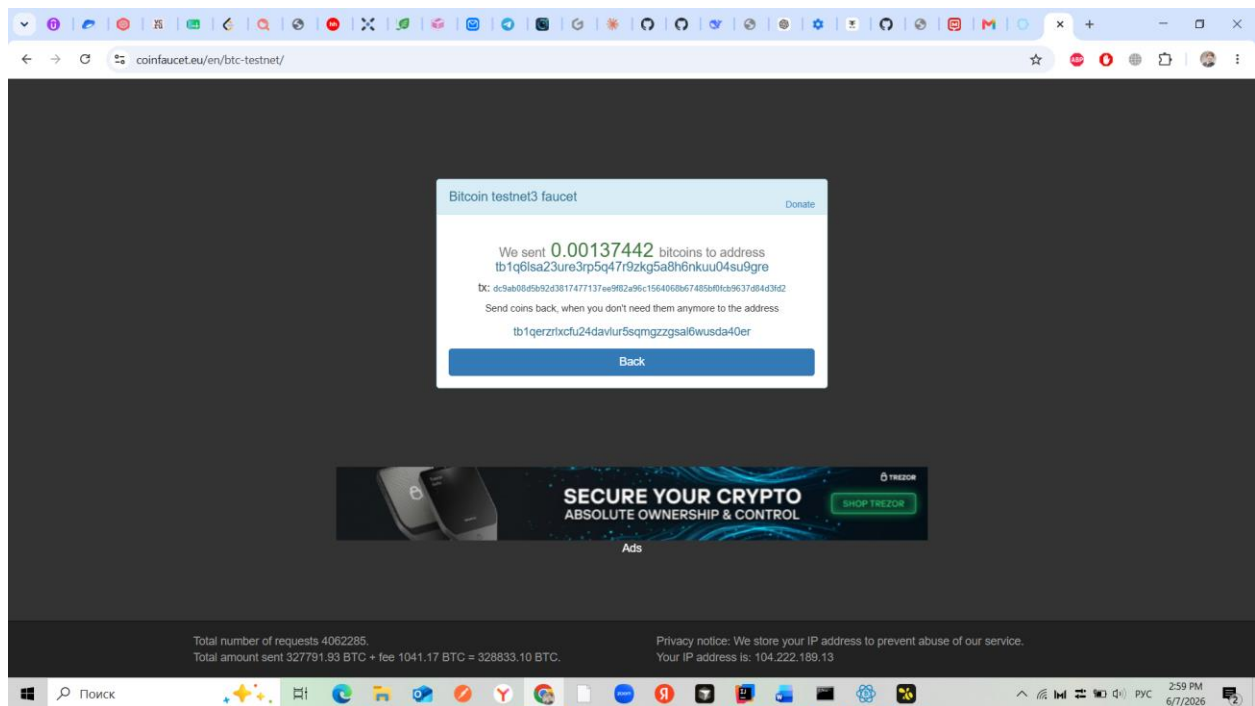
Node.js v24.14.1
PS C:\Users\Евгения\otus-homework\blockchain-project> node bitcoin/level1/latest_block.js
=== Bitcoin Testnet - Информация о последнем блоке ===
[x] Хэш последнего блока: 000000000000028асес3611а5с9а5740bc37е33ba7341b892е1f83ае2а95b809
[x] Высота блока: 4968765
[x] Время создания блока: 06.06.2026, 16:50:15
[x] Количество транзакций в блоке: 14
[x] Размер блока: 8546 байт (8.35 KB)

[x] Список первых 10 транзакций (TXID) из 14:
1. d68fbacedf4dab47de3ab3183b3f8eae2bdbbf8e2f3c8d4d083d2a080f0137582
2. 45b5d484dce6bc6a68f1a2bb706e930f35391c2938addb162f58ab8ed8272e7
3. 48cdf56cc1151a98a584f98a7dca36de5f02dd5a6156ebef378a2e0da6269b71
4. 050e0b507958fbdcd5d177fb50f5b3283bb7bc871ed33b2a1af815d3d397a5df5
5. bdf2289616fd95bd497f63d270222278ae8a27b7bb3f20a8fc3b76906c746c33
6. 5bc1af9194aa113194a7778df61019dc29099912fafde78006337f87caffff5d
7. c040801c1f6dac5c6a785bc71171504ce2d1c3498729008f3947440f8e83e2fb
8. d1804ae79780729f9d6bb26c6001b457f6a4f49bb2518b1f400e1ad85e1a8476
9. d188aac03390788сес580005bc041980e00e88b74484854f5d2771fb741bc72c
10. 2612f32f0801bcf5ead1c2391469505ee79ea8fb96dce5143b826e307f7f7e3a

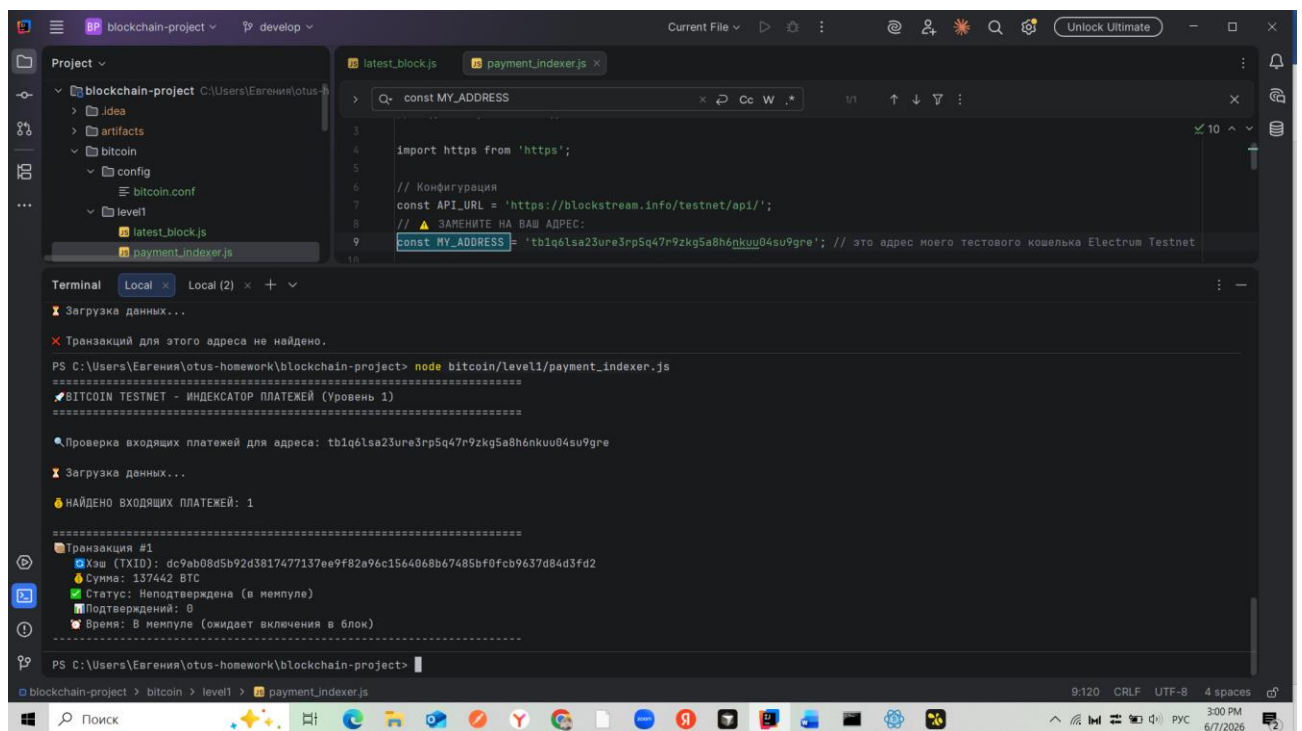
```

4. Разработка индексатора платежей

- Напишите скрипт для проверки входящих платежей на **testnet-адрес**.
- Выводите: сумма, хэш транзакции, число подтверждений.
- Зашла на сайт-кран coinfaucet.eu
- сделала запрос на отправку на мой кошелек тестовых монет



- Проверила свой скрипт, который получает информацию из сети о входящих транзакциях на мой кошелек:



```
const MY_ADDRESS

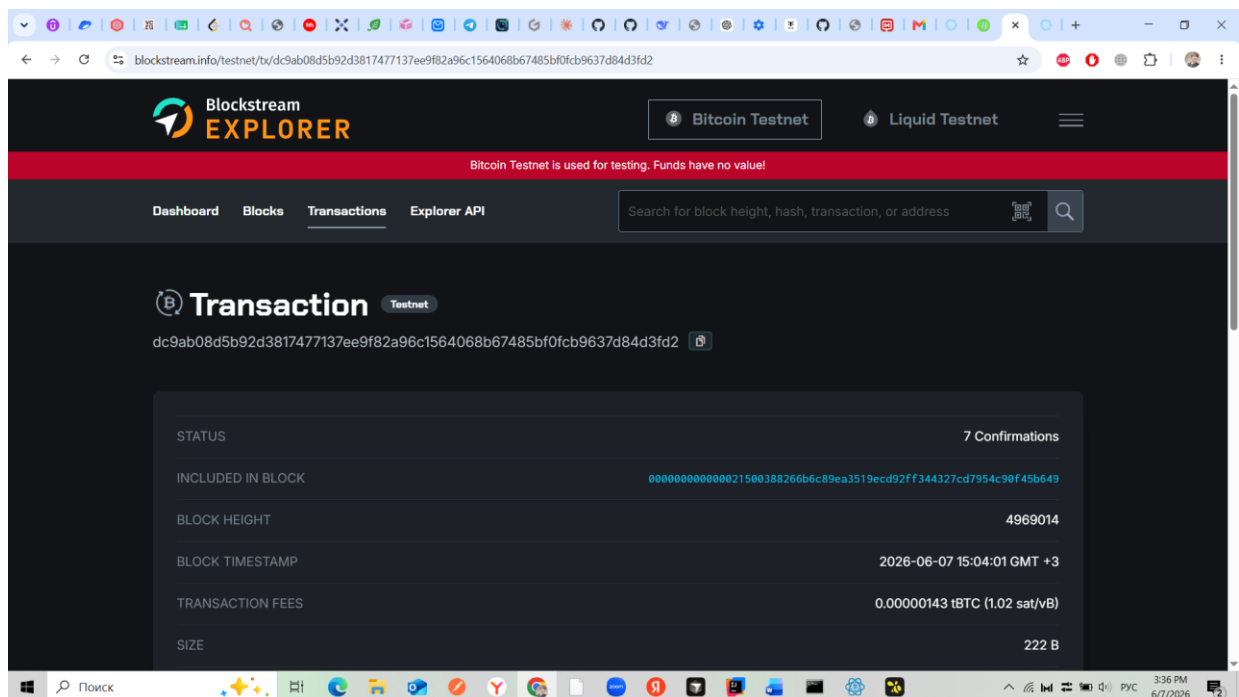
import https from 'https';

// Конфигурация
const API_URL = 'https://blockstream.info/testnet/api/';
// ЗАМЕНИТЕ НА ВАШ АДРЕС:
const MY_ADDRESS = 'tb1q61sa23ure3rp5q47r9zkg5a8h6nkuu04su9gre'; // это адрес моего тестового кошелька Electrum Testnet

Terminal
Подтверждений: 0
Время: В мейнпеле (ожидает включения в блок)

PS C:\Users\Eвгения\otus-homework\blockchain-project> node bitcoin\level1\payment_indexer.js
=====
BITCOIN TESTNET - ИНДЕКСАТОР ПЛАТЕЖЕЙ (Уровень 1)
=====
Проверка входящих платежей для адреса: tb1q61sa23ure3rp5q47r9zkg5a8h6nkuu04su9gre
Загрузка данных...
НАЙДЕНО ВХОДЯЩИХ ПЛАТЕЖЕЙ: 1
Транзакция #1
Хэш (TXID): dc9ab08d5b92d3817477137ee9f82a96c1564068b67485bf0fcb9637d84d3fd2
Сумма: 137442 BTC
Статус: Подтверждена
Подтверждений: 4
Время: 07.06.2026, 15:04:01
=====
PS C:\Users\Eвгения\otus-homework\blockchain-project>
```

- Нашла транзакцию, отправленную сайт-краном (coinfaucet.eu) на мой кошелек:
<https://blockstream.info/testnet/tx/dc9ab08d5b92d3817477137ee9f82a96c1564068b67485bf0fcb9637d84d3fd2>



5. Оптимизация и анализ

- Уровень 1:** исследуйте ограничения API и предложите способы оптимизации (кеширование, лимиты, обработка ошибок).

Ограничения API Blockstream Esplora

Ограничение	Описание
Rate limiting	Негласные лимиты (~10-20 запросов/сек), при превышении возвращается ошибка 429
Нет WebSocket/ZMQ	Отсутствует возможность подписки на события в реальном времени → только polling
Ограниченная история	Не все старые транзакции доступны через API
Приватность	Провайдер видит все запросы (какие адреса проверяются)
Нет RPC-методов	Недоступны getnetworkinfo, getpeerinfo и другие команды для анализа сети

Предлагаемые способы оптимизации

Оптимизация	Как реализовать	Эффект
Кеширование	Сохранять результат запроса blocks/tip/hash на 10-30 секунд	Снижение числа запросов в 5-10 раз
Пагинация	Использовать ?limit=25 для загрузки только первых N транзакций	Ускорение работы, снижение нагрузки
Retry logic	Повторные попытки при ошибках с экспоненциальной задержкой (1, 2, 4 секунды)	Повышение надёжности при временных сбоях
Таймауты	Установить лимит ожидания ответа (30 секунд)	Предотвращение зависания программы
Polling-интервал	Опрашивать адрес не чаще 1 раза в 15-30 секунд	Снижение нагрузки на API и сеть

Рефлексия

1. Какие трудности возникли при выполнении задания?

Трудность	Как было решено
Нехватка дискового пространства для полной ноды	Выбран гибридный подход: Level 1 (публичное API) + частичная настройка ноды с <code>prune=550</code>
Ошибка Cannot find module при запуске скрипта	Неправильный путь к файлу — исправлен на <code>node bitcoin/level1/latest_block.js</code>
Отсутствие работающих кранов для testnet	Перепробовал несколько вариантов, в итоге сработал <code>coinfaucet.eu</code>
Сообщение "Your account does not allow you to access the faucet"	Обнаружил кризис тестовой сети Bitcoin в 2026 году, перешёл на альтернативный кран

2. Что нового вы узнали о работе сети Bitcoin?

Что узнал(а)	Подробнее
Структура блока	Блок содержит: хэш, высоту, время, merkle root, список транзакций (TXID), размер, вес
Mempool	Очередь неподтверждённых транзакций; у каждого узла свой mempool; транзакции сортируются по комиссии, в тестовой среде нужно ждать минут 15, прежде чем увижу мою транзакцию в блоке
Testnet vs Mainnet	Тестовая сеть имитирует реальную, но монеты не имеют ценности; нужна для разработки и отладки
Pruning	Режим экономии места: нода сначала валидирует все блоки, затем удаляет старые, оставляя только UTXO-набор. Также: значение <code>prune</code> не решает проблему нехватки ресурсов на свое ПК, т.к. какое бы малое значение <code>prune</code> не было указано, сначала все-равно вычитываются все имеющиеся в сети блоки. И только после этого они 'обрезаются' до значения, указанного в <code>prune</code>
RPC vs REST API	RPC — прямой доступ к своей ноде (полный контроль), REST API — доступ к чужому серверу (проще, но ограничения)

3. Чем отличается опыт использования публичного API и собственной ноды?

Критерий	Публичное API (Level 1)	Собственная нода (Level 2)
Ресурсы	Не требует дискового пространства	~500+ ГБ (или ~2-3 ГБ с prune=550)
Время настройки	Минуты (выбрать API, написать запрос)	Часы/дни (синхронизация блокчейна)
Скорость запросов	Зависит от интернета и загрузки сервера	Мгновенная (локальный вызов)
Приватность	Низкая — провайдер видит все запросы	Полная — никто не знает, какие адреса вы проверяете
Rate limits	Есть (негласные, ~10-20 запросов/сек)	Нет
Real-time события	Только polling (постоянные запросы)	ZMQ — подписка на события в реальном времени